Vektör Uzayları

Konu Anlatımı, Adım Adım Soru Çözümleri, Grafikler ve İlgili Kodlar

Mühendislik Matematiği, 4.1 ve 4.2

Elif Nur Öksüz

Öğrenci No: 258102028

Giriş

Bu çalışmada 4.1 ve 4.2 konularını (vektör uzayı, alt uzay, doğrusal germe, bağımsızlık ve baz) Python ile yazılmış bir web uygulaması üzerinden anlattım. Belgedeki grafikler ve matris görüntüleri uygulamanın ekranından alındı. Hesapların hepsi kesirlerle (Fraction) yapıldığı için yuvarlama hatası yok.

Uygulamayı çalıştırmak için:

Terminal

```
cd vektor_uzaylari_web
py -3 app.py
# tarayıcıda: http://127.0.0.1:8000
```

Her konuda önce kısa bir anlatım, sonra o konuyla ilgili kod, en son da derslerin adımları var. Derslerde ispat satırları genel yazıldı, grafik sadece bir örnek. Bir "her … için" iddiası tek örnekle ispatlanamaz ama tek bir karşı örnekle çürütülebilir.

4.1 Vektör Uzayları

Tanım 4.1.1. V boş olmayan bir küme, F bir cisim (bizde $\mathbb{R}$) olsun. V üzerinde bir **toplama** (V×V→V) ve bir **skalerle çarpma** (F×V→V) tanımlı ve aşağıdaki sekiz kural her u, v, w ∈ V ve her a, b, c ∈ F için sağlanıyorsa V'ye **vektör uzayı** denir.

Kapalılık (iki vektörün toplamı ve bir vektörün katı yine V'de) işlemlerin tanımının parçasıdır; ayrı kart olarak yazılmadı.

Vektör sadece ok değildir: $\mathbb{R}^n$, m×n matrisler, derecesi en fazla n olan polinomlar (P_n) ve fonksiyonlar (ör. sinyaller) da bu kuralları sağlar. Mühendislikte süperpozisyon, kuvvetlerin toplanması, sinyal karıştırma hep bu yapıyı kullanır.

Örnek mi, ispat mı?

- **∀ (her … için)** iddiası bir örnekle ispatlanamaz: sonsuz çok durumdan yalnızca birini kontrol etmiş oluruz. Keyfi (genel) elemanlarla cebirsel bir argüman gerekir.
- Aynı iddia tek bir **karşı örnekle çürütülür**: ¬(∀x P(x)) ⇔ ∃x ¬P(x).
- Bu uygulamada grafikler ispatın resmidir; ispatın kendisi soldaki gerekçeli satırlardır. "Başka sayılarla dene" düğmesiyle sayıları değiştir: satırlar değişmez.

core/arithmetic.py (satır 101–107)

```python
def vector_add(a: Sequence[Fraction], b: Sequence[Fraction]) -> List[Fraction]:
    if len(a) != len(b):
    sonuc: List[Fraction] = []
    for i in range(len(a)):
        sonuc.append(a[i] + b[i])
    return sonuc
```

core/arithmetic.py (satır 116–117)

```python
def scalar_mul(scalar: Fraction, vec: Sequence[Fraction]) -> List[Fraction]:
    return [scalar * v for v in vec]
```

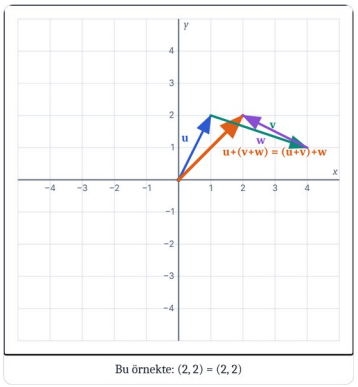
core/arithmetic.py (satır 120–134)

```python
def linear_combination(
    vectors: Sequence[Sequence[Fraction]], coeffs: Sequence[Fraction]
) -> List[Fraction]:
    if len(vectors) != len(coeffs):
    boyut = len(vectors[0])
    sonuc = [Fraction(0) for _ in range(boyut)]
    for vec, k in zip(vectors, coeffs):
        if len(vec) != boyut:
        for i in range(boyut):
            sonuc[i] += k * vec[i]
    return sonuc
```

vector_add: iki vektörü bileşen bileşen toplar ($u_i + v_i$). V1 ve V4 aksiyomlarının ispatındaki "bileşen bileşen" adımı tam olarak budur. scalar_mul: her bileşeni aynı skalerle çarpar ($c \cdot v_i$). V5-V8 bu işleme dayanır. linear_combination: $c_1v_1 + \ldots + c_kv_k$ toplamını sıfır vektöründen başlayarak biriktirir. Bütün sayılar Fraction olduğu için yuvarlama hatası olmaz.

V1: Toplamada birleşme

İddia: Her u, v, w ∈ $\mathbb{R}^n$ için $u + (v + w) = (u + v) + w$



Bu örnekte: (2, 2) = (2, 2)

1. Neyi ispatlıyoruz? İddia **her** u, v, w için geçerli olmalı. “Her … için” diye başlayan bir iddia tek bir sayısal örnekle ispatlanamaz; bu yüzden **keyfi** (herhangi) vektörlerle, bileşen bileşen çalışacağız.

$u = (u_1, \ldots, u_n), \quad v = (v_1, \ldots, v_n), \quad w = (w_1, \ldots, w_n)$

Gerekçe: Keyfi seçim: hiçbir özel sayı kullanılmıyor.

2. Sol taraf, 1. adım: v + w: Parantez önce yapılır. Vektör toplamı uç uca ekleme demektir: w, v'nin ucundan başlar.

$v + w = (v_1 + w_1, \ldots, v_n + w_n)$

Gerekçe: Vektör toplamının tanımı (bileşen bileşen).

3. Sol taraf, 2. adım: u + (v + w): Şimdi (v + w) okunu u'nun ucuna taşıyoruz.

$[u + (v + w)]_i = u_i + (v_i + w_i)$

Gerekçe: Toplamın tanımı, i. bileşen için.

4. Sağ taraf: (u + v) + w: Bu kez önce u + v, sonra w ekleniyor. Farklı bir yoldan yürüdük; uç noktaya bak.

$[(u + v) + w]_i = (u_i + v_i) + w_i$

Gerekçe: Toplamın tanımı, i. bileşen için.

5. Kilit adım: İki tarafın i. bileşenleri artık sadece **gerçek sayılardır**. Gerçek sayıların toplaması birleşmeli olduğundan bunlar eşittir.

$u_i + (v_i + w_i) = (u_i + v_i) + w_i \quad (\text{her } i = 1, \ldots, n)$

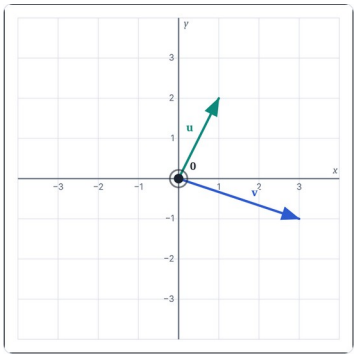
Gerekçe: $\mathbb{R}$'de toplama birleşmelidir.

6. Sonuç: Bütün bileşenleri eşit olan iki vektör eşittir. İspatta u, v, w'nin **hangi sayılar olduğunu hiç kullanmadık**; yalnızca $\mathbb{R}$'nin birleşme özelliğini kullandık. Bu yüzden iddia her vektör için doğrudur. "Başka sayılarla dene" düğmesiyle başka vektörler seç: adımlar aynen geçerli kalır.

$u + (v + w) = (u + v) + w$

V2: Sıfır vektörü

İddia: Öyle bir 0 vardır ki her v için $v + 0 = 0 + v = v$



1. İddianın yapısı: Bu bir **varlık** iddiası: “öyle bir 0 vardır”. Varlığı göstermenin en temiz yolu adayı açıkça vermek ve şartı sağladığını kontrol etmektir.

Aday: $0 = (0, 0, \ldots, 0)$

Gerekçe: Varlık ispatı: adayı inşa et, sonra doğrula.

2. v + 0 = v: v'nin ucuna uzunluğu sıfır olan bir ok eklemek hiçbir yere götürmez.

$[v + 0]_i = v_i + 0 = v_i$

Gerekçe: 0, $\mathbb{R}$'de toplamanın birim elemanıdır.

3. 0 + v = v:

$[0 + v]_i = 0 + v_i = v_i$

Gerekçe: Aynı gerekçe (ve $\mathbb{R}$'de değişme).

4. Aynı 0 herkes için çalışır: Aday 0, v'ye bağlı değildir: başka bir u için de u + 0 = u. Aksiyom, her vektörün “kendi sıfırı” olmasını değil, **tek ortak** bir sıfır olmasını ister.

$[u + 0]_i = u_i + 0 = u_i$

5. sıfır vektörü tektir: 0 ve 0′ iki sıfır vektörü olsun. Teklik ispatının standart kalıbı: ikisini birbirine eşitle.

$0' = 0' + 0 \quad (0 \text{ sıfır vektörü})$

$0' + 0 = 0 \quad (0' \text{ sıfır vektörü})$

$\Rightarrow \; 0' = 0$

Gerekçe: Sadece V2 kullanıldı; bileşen gerekmedi, her vektör uzayında geçerli.

6. Sonuç: 0 = (0, …, 0) her vektör için şartı sağladı ve başka bir sıfır olamaz.

$v + 0 = 0 + v = v, \;$ her v için

V3: Toplamsal ters

İddia: Her v için öyle bir −v vardır ki $v + (-v) = (-v) + v = 0$

1. **Her v için bir ters bul:** Bu kez sıra önemli: önce v keyfi verilir, **sonra** ona bağlı bir −v kurulur (her vektörün kendi tersi vardır; sıfırın aksine).

$v = (v_1, \ldots, v_n)$ keyfi

2. **Adayı inşa et:** Grafikte −v, v ile aynı uzunlukta ve tam zıt yönde.

$-v = (-v_1, \ldots, -v_n)$

Gerekçe: Her gerçek sayının toplamsal tersi vardır.

3. **v + (−v) = 0:** −v'yi v'nin ucuna taşı: ok bizi tam orijine geri getirir.

$[v + (-v)]_i = v_i + (-v_i) = 0$

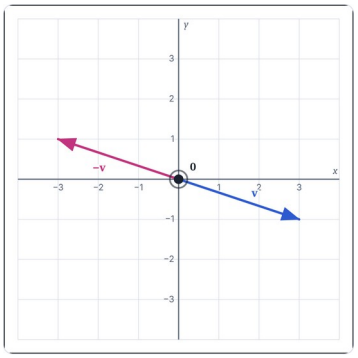
Gerekçe: $\mathbb{R}$'de $x + (-x) = 0$.

4. **(−v) + v = 0:**

$[(-v) + v]_i = -v_i + v_i = 0$

5. **Sonuç:** Her v için −v'yi açıkça kurduk ve şartı sağladığını bileşen bileşen gösterdik. Not: −v'nin (−1)·v'ye eşit olduğu bir aksiyom değil, **teoremdir**; "Teorem 4.1.1" derslerinde aksiyomlardan ispatlanıyor.

$v + (-v) = (-v) + v = 0$



V4: Toplamada değişme

İddia: Her $u, v \in \mathbb{R}^n$ için $u + v = v + u$

1. **Keyfi iki vektör:**

$u = (u_1, \ldots, u_n), \quad v = (v_1, \ldots, v_n)$

Gerekçe: Keyfi seçim.

2. **1. yol: önce u, sonra v:**

$[u + v]_i = u_i + v_i$

Gerekçe: Toplamın tanımı.

3. **2. yol: önce v, sonra u:** Diğer kenarlardan yürüyünce bir **paralelkenar** oluşur.

$[v + u]_i = v_i + u_i$

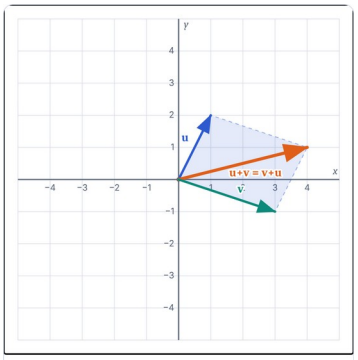
4. **Kilit adım:** Paralelkenarın iki yolu aynı köşeye çıkar: köşegen = u + v = v + u.

$u_i + v_i = v_i + u_i \quad$ (her i için)

Gerekçe: $\mathbb{R}$'de toplama değişmelidir.

5. **Sonuç:** Bileşenler eşit ⇒ vektörler eşit. Sayılar özel seçilmedi; "Başka sayılarla dene" düğmesiyle dene.

$u + v = v + u$



Bu örnekte: (4, 1) = (4, 1)

V5: Vektör toplamına dağılma

İddia: Her $c \in \mathbb{R}$ ve $u, v \in \mathbb{R}^n$ için $c(u + v) = cu + cv$

1. **Kurulum:**

$c \in \mathbb{R}$ keyfi, $u, v \in \mathbb{R}^n$ keyfi

2. **Sol taraf: önce topla, sonra c ile çarp:**

$[c(u + v)]_i = c(u_i + v_i)$

Gerekçe: Toplamın ve skaler çarpmanın tanımı.

3. **Sağ taraf: önce çarp, sonra topla:** Bütün üçgen c oranında büyüdü (c < 0 ise ayrıca ters döndü): **benzer üçgenler**.

$[cu + cv]_i = cu_i + cv_i$

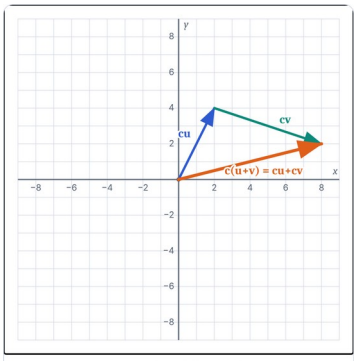
4. **Kilit adım:**

$c(u_i + v_i) = cu_i + cv_i \quad$ (her i için)

Gerekçe: $\mathbb{R}$'de çarpma toplama üzerine dağılır.

5. **Sonuç:**

$c(u + v) = cu + cv$

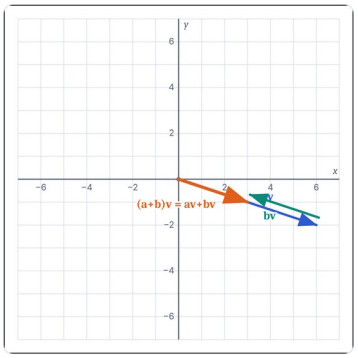


Bu örnekte: (8, 2) = (8, 2)

V6: Skaler toplamına dağılma

İddia: Her $a, b \in \mathbb{R}$ ve $v \in \mathbb{R}^n$ için $(a + b)v = av + bv$

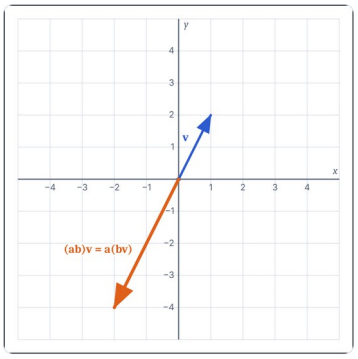
1. **Kurulum:**
   $a, b \in \mathbb{R}$ keyfi, $v \in \mathbb{R}^n$ keyfi
2. **av:**
3. **av'nin ucuna bv ekle:** Hepsi aynı doğru üzerinde; görünsün diye bv okunu biraz yana kaydırdık.
4. **(a + b)v ile karşılaştır:**
   $[(a + b)v]_i = (a + b)v_i$
   $[av + bv]_i = av_i + bv_i$
   $(a + b)v_i = av_i + bv_i$
   Gerekçe: $\mathbb{R}$'de dağılma özelliği.
5. **Sonuç:** Bu aksiyomun bozulduğu bir yapıyı "Karşı örnek: garip çarpma" dersinde görebilirsin.
   $(a + b)v = av + bv$



V7: Skaler çarpımının uyumu

İddia: Her $a, b \in \mathbb{R}$ ve $v \in \mathbb{R}^n$ için $(ab)v = a(bv)$

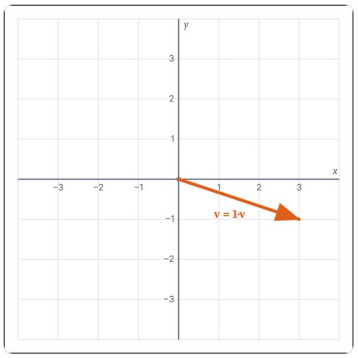
1. **Kurulum:**
   $a, b \in \mathbb{R}$, $v \in \mathbb{R}^n$ keyfi
2. **Önce b ile çarp:**
   $[bv]_i = bv_i$
3. **Sonra a ile çarp:**
   $[a(bv)]_i = a(bv_i)$
4. **Tek seferde ab ile çarp:**
   $[(ab)v]_i = (ab)v_i = a(bv_i)$
   Gerekçe: $\mathbb{R}$'de çarpma birleşmelidir.
5. **Sonuç:** "İki kez ölçeklemek = çarpımla bir kez ölçeklemek." Mühendislikte ardışık kazanç (ör. iki yükselteç) bu kuraldır.
   $(ab)v = a(bv)$



V8: Birim skaler

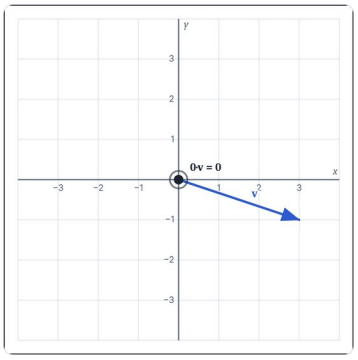
İddia: Her v için $1 \cdot v = v$

1. **Bileşen bileşen:**
   $[1 \cdot v]_i = 1 \cdot v_i = v_i$
   Gerekçe: 1, $\mathbb{R}$'de çarpmanın birim elemanıdır.
2. **Bu aksiyom neden ayrıca yazılıyor?** Skaler çarpmayı "her şeyi sıfıra götür" diye tanımlayalım: $c \odot v = 0$. O zaman V1-V7'nin hepsi sağlanır (iki taraf da 0 olur), ama $1 \odot v = 0 \neq v$. Demek ki V8 diğerlerinden **çıkmaz**; kural listesinden atılamaz.
   $c \odot (u + v) = 0 = 0 + 0 = c \odot u + c \odot v$ (V5)
   $1 \odot v = 0 \neq v$ (V8 sağlanmaz)
   Gerekçe: Bir aksiyomun gerekli olduğunu göstermek için: onun dışındakileri sağlayan ama onu bozan bir model bul.
3. **Sonuç:**
   $1 \cdot v = v$



Teorem 4.1.1 (a): 0·v = 0

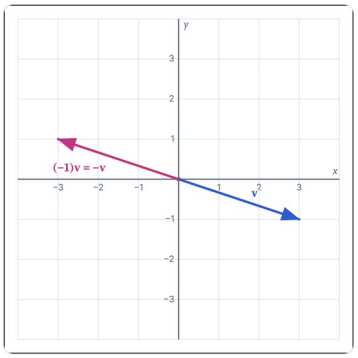
İddia: Her vektör uzayında, her v için $0 \cdot v = 0$



1. **Bileşen kullanmadan ispatlayacağız:** Bileşenle ispat sadece $\mathbb{R}^n$'de çalışır. Bu ispat **yalnızca aksiyomları** kullanacak; böylece polinomlarda, matrislerde, fonksiyonlarda, her vektör uzayında, geçerli olacak. Her satırın yanında hangi kuralın kullanıldığı yazıyor.
2. **Sıfırı ikiye böl:**
   $0\cdot v = (0+0)\cdot v$
   Gerekçe: $\mathbb{R}$'de 0 = 0 + 0.
3. **Dağıt:**
   $(0+0)\cdot v = 0\cdot v + 0\cdot v$
   Gerekçe: V6 (skaler toplamına dağılma).
4. **Elimizdeki denklem:** Bir vektör kendisinin iki katına eşit çıktı. Şimdi iki tarafa x'in tersini ekleyeceğiz.
   $x = x + x, \quad \text{burada } x = 0\cdot v$
   Gerekçe: V3: −x vardır.
5. **İki tarafa −x ekle:**
   $x + (-x) = (x + x) + (-x)$
   $0 = x + (x + (-x))$
   $0 = x + 0 = x$
   Gerekçe: 1. satır: V3 ve eşitlik · 2. satır: V3 (sol), V1 (sağ) · 3. satır: V3, V2
6. **Sonuç:** 0·v = 0. Kullanılanlar: V1, V2, V3, V6 ve $\mathbb{R}$'nin 0 = 0 + 0 özelliği. Hiçbir koordinat kullanılmadı.
   $0\cdot v = 0$

Teorem 4.1.1 (b): (−1)v = −v

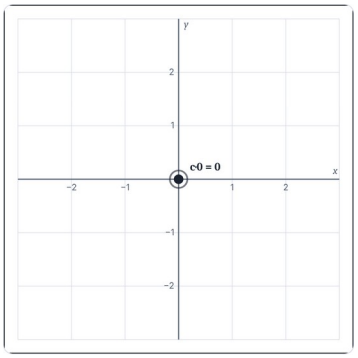
İddia: Her vektör uzayında $(-1)\cdot v = -v$



1. **Fikir:** −v, v'ye eklenince 0 veren vektördür (V3). (−1)v'nin de bu işi yaptığını gösterip, tersin **tek** olduğunu kullanacağız.
2. **v'yi 1·v diye yaz:**
   $v + (-1)v = 1\cdot v + (-1)\cdot v$
   Gerekçe: V8.
3. **Skalerleri topla:**
   $1\cdot v + (-1)\cdot v = (1 + (-1))\cdot v = 0\cdot v$
   Gerekçe: V6 ve $\mathbb{R}$'de 1 + (−1) = 0.
4. **Önceki teorem:**
   $0\cdot v = 0$
   Gerekçe: Teorem 4.1.1 (a).
5. **Ters tektir:** x + v = 0 ve y + v = 0 olsun. O zaman:
   $x = x + 0 = x + (v + y) = (x + v) + y = 0 + y = y$
   Gerekçe: V2, V4, V1, V2. Yani v'nin iki farklı tersi olamaz.
6. **Sonuç:** (−1)v, v'nin bir tersidir ve ters tek olduğundan (−1)v = −v.
   $(-1)\cdot v = -v$

Teorem 4.1.1 (c): c·0 = 0

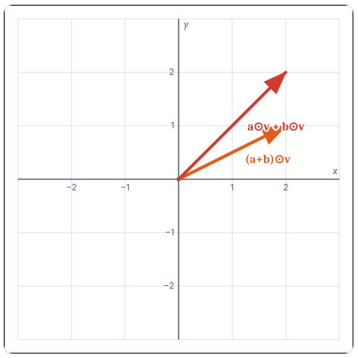
İddia: Her c skaleri için $c\cdot 0 = 0$



1. **0 = 0 + 0 (vektör olarak):**
   $c\cdot 0 = c\cdot(0 + 0)$
   Gerekçe: V2: 0 + 0 = 0.
2. **Dağıt:**
   $c\cdot(0 + 0) = c\cdot 0 + c\cdot 0$
   Gerekçe: V5.
3. **Aynı kalıp: x = x + x ⇒ x = 0:**
   $x = c\cdot 0 \text{ için } x = x + x$
   $\Rightarrow x = 0 \quad \text{(iki tarafa } -x \text{ ekle)}$
   Gerekçe: V3, V1, V2 (önceki ispattaki gibi).
4. **Sonuç:** Aynı ispat kalıbını (x = x + x ⇒ x = 0) iki kez kullandık: matematikte iyi bir fikir tekrar tekrar kullanılır.
   $c\cdot 0 = 0$

Karşı örnek: vektör uzayı olmayan bir yapı

İddia: $\mathbb{R}^2$'de toplama olağan, ama skaler çarpma $c \odot (x, y) = (cx, y)$ olsun. Vektör uzayı mı?



1. Yeni bir "çarpma" tanımı: Bu çarpma yalnızca x bileşenini ölçekliyor. Sekiz kuralın hepsini sağlıyor mu? Şüpheli olan V6: $(a + b)\odot v = a\odot v + b\odot v$.

$$c \odot (x, y) = (cx, y)$$

2. Sol taraf: $(a + b) \odot v$:

$$(a + b) \odot (x, y) = ((a + b)x, y)$$

3. Sağ taraf: $a \odot v + b \odot v$:

$$(ax, y) + (bx, y) = ((a + b)x, 2y)$$

4. Uçlar farklı! $y \neq 0$ olduğu sürece $2y \neq y$. "Her a, b, v için" iddiasını **çürütmek** için tek bir karşı örnek yeter: $a = 1$, $b = 1$, $v = (1, 1)$.

$$(2, 1) \neq (2, 2)$$

Gerekçe: Mantık: $\neg(\forall x\, P(x)) \Leftrightarrow \exists x\, \neg P(x)$.

5. Sonuç: V6 sağlanmadığı için $(\mathbb{R}^2, +, \odot)$ bir vektör uzayı **değildir**. İspat ile çürütme arasındaki asimetriye dikkat: doğruluğu göstermek için genel bir argüman, yanlışlığı göstermek için tek bir örnek gerekir.

4.1.1 Alt Uzaylar

Tanım 4.1.2-4.1.3. V bir vektör uzayı, W ⊆ V boş olmayan bir alt küme olsun. W, V'nin **aynı işlemleriyle** kendisi de bir vektör uzayıysa W'ye **alt uzay** denir. Pratikte üç koşul yeter:

Koşul	Soru	Nasıl gösterilir?
1. Sıfır vektörü	$0 \in W$ mi?	Yerine koy.
2. Toplamaya kapalılık	$u, v \in W \Rightarrow u + v \in W$?	Sağlanıyorsa: keyfi u, v ile genel cebir. Sağlanmıyorsa: tek karşı örnek.
3. Skalerle çarpmaya kapalılık	$u \in W, \lambda \in \mathbb{R} \Rightarrow \lambda u \in W$?	Aynı mantık.

Neden önce sıfır? W alt uzaysa herhangi $u \in W$ için $0 \cdot u = 0 \in W$ olmak zorundadır. $0 \notin W$ ise hüküm hemen verilir; $0 \in W$ olması ise tek başına yetmez.

Mühendislik gözüyle

Ax = 0 (homojen) sistemlerin ve **y″ + y = 0** gibi doğrusal homojen diferansiyel denklemlerin çözüm kümeleri alt uzaydır: süperpozisyon ilkesi tam olarak budur.

Ax = b ($b \neq 0$) çözümleri alt uzay değildir (0 çözüm değil), ama "özel çözüm + alt uzay" biçimindedir.

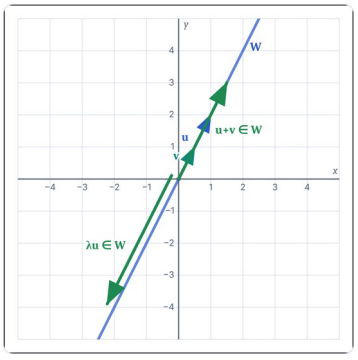
core/lessons.py (satır 174–187)

```
def _in_set(kume: Optional[dict], p) -> Optional[bool]:
    if not kume or not is_vector(p) or len(p) != 2:
        return None
    x, y = p
    if kume["type"] == "line":
        return y == Fraction(kume["m"]) * x + Fraction(kume.get("b", "0"))
    bolge = kume.get("region")
    if bolge == "q1":
        return x > 0 and y > 0
    if bolge == "q13":
        return x * y >= 0
    if bolge == "all":
        return True
    return None
```

Bir noktanın kümeye (W) ait olup olmadığını kesin aritmetikle söyler; grafikteki yeşil/kırmızı renklendirme buradan gelir. "line" türü: $y = m \cdot x + b$ doğrusu ($b = 0$ ise alt uzay, $b \neq 0$ ise sıfır vektörü yok). "q1": $x > 0$ ve $y > 0$ bölgesi; "q13": $xy \geq 0$ (1. ve 3. bölge). Karşı örnekler bu fonksiyonla "küme dışında" diye işaretlenir.

y = 2x: genel ispatla alt uzay

İddia: $W = \{(x, y) \in \mathbb{R}^2 : y = 2x\}$ bir alt uzaydır.

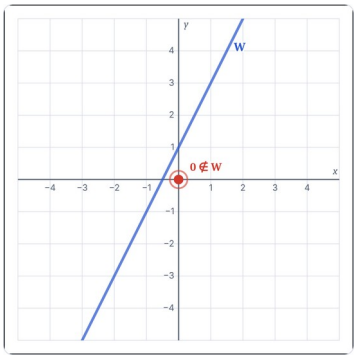


1. **Kümeyi parametreyle yaz:** W'nin her elemanı (t, 2t) biçimindedir. Üç koşulu **her eleman ve her skaler** için göstereceğiz; sayısal deneme yapmayacağız.
   $W = \{(t, 2t) : t \in \mathbb{R}\}$
2. **1) Sıfır vektörü:**
   $t = 0 \Rightarrow (0, 0) \in W \quad (0 = 2 \cdot 0)$
3. **2) Toplama: keyfi iki eleman:**
   $u = (r, 2r), \quad v = (s, 2s) \quad (r, s \text{ keyfi})$
4. **2) Toplama: topla ve biçime bak:**
   $u + v = (r + s, 2r + 2s) = (r + s, 2(r + s))$
   Gerekçe: Dağılma: $2r + 2s = 2(r + s)$. $t = r + s$ alınca $(t, 2t)$ biçimi.
5. **3) Skalerle çarpma:**
   $\lambda u = (\lambda r, \lambda \cdot 2r) = (\lambda r, 2(\lambda r))$
   Gerekçe: $\mathbb{R}$'de çarpma birleşmeli ve değişmeli. $t = \lambda r$ alınca yine $(t, 2t)$.
6. **Sonuç:** Üç koşul **her** r, s, λ için sağlandı ⇒ W bir alt uzaydır. Grafikteki sayılar ispatın parçası değildir: 'e bas, aynı satırlar yine doğru.

$W \leq \mathbb{R}^2$

y = 2x + 1: sıfır yok

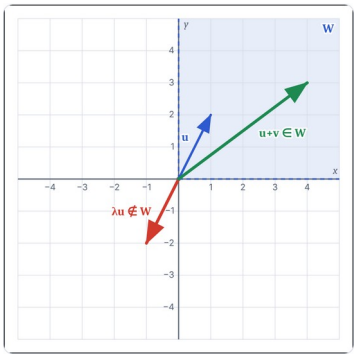
İddia: W = {(x, y) : y = 2x + 1} alt uzay mı?



1. **Küme:** y = 2x doğrusunu bir birim yukarı kaydırdık.
   $W = \{(t, 2t + 1) : t \in \mathbb{R}\}$
2. **Neden önce sıfıra bakıyoruz?** Her alt uzay 0'ı içermek **zorundadır**: W boş değilse bir u ∈ W alırız; skalerle kapalılıktan 0·u ∈ W, ve Teorem 4.1.1'e göre 0·u = 0. Yani "0 ∉ W" tek başına hükmü verir.
   $u \in W \Rightarrow 0 \cdot u = 0 \in W$
3. **1) Sıfır vektörü:**
   $0 = 2 \cdot 0 + 1 = 1$ sağlanmaz
4. **Hüküm:** Bir koşulun bozulması yeter ⇒ W alt uzay **değildir**.
5. **Ek: toplama da HER zaman bozuluyor (genel):**
   $u = (r, 2r + 1),\ v = (s, 2s + 1)$
   $u + v = (r + s, 2(r + s) + 2)$
   W'de olması için $2 = 1$ gerekirdi ⇒ hiçbir r, s için u + v ∈ W değil
   Gerekçe: Bu kez genel bir argümanla "hiçbir zaman" sonucunu çıkardık.
6. **Mühendislik bağlantısı:** y = 2x + 1 ⇔ 2x − y = −1: **homojen olmayan** bir denklem. Ax = b (b ≠ 0) sistemlerinin çözüm kümesi asla alt uzay değildir, çünkü A·0 = 0 ≠ b. Ayrıntı: "Homojen sistemler" dersi.
   $A \cdot 0 = 0 \neq b$

x > 0 ve y > 0: toplama yetmez

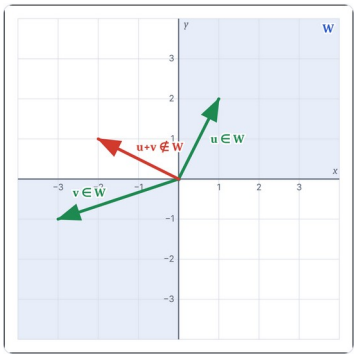
İddia: W = {(x, y) : x > 0 ve y > 0} alt uzay mı?



1. **Küme: birinci bölgenin içi:** Eksenler dahil değil.
2. **1) Sıfır vektörü:** Hüküm zaten verildi (alt uzay değil). Yine de hangi koşulların sağlandığını görmek öğretici.
   $0 > 0$ yanlış ⇒ $(0, 0) \notin W$
3. **2) Toplama: genel olarak sağlanır:**
   $x_1, x_2 > 0 \Rightarrow x_1 + x_2 > 0$
   $y_1, y_2 > 0 \Rightarrow y_1 + y_2 > 0$
   Gerekçe: Pozitif sayıların toplamı pozitiftir (her seçim için).
4. **3) Skalerle çarpma: karşı örnek:** "Her λ için" iddiasını tek bir λ çürütür.
   $\lambda = -1$: $\lambda u = (-1, -2)$
   Gerekçe: Negatif sayıyla çarpınca bileşenler negatif olur.
5. **Sonuç:** Toplamaya kapalı ama skalerle çarpmaya kapalı değil (ve 0 yok) ⇒ alt uzay **değil**. Ders: tek bir koşulun sağlanması yetmez.

xy ≥ 0: sıfır var, yine de değil

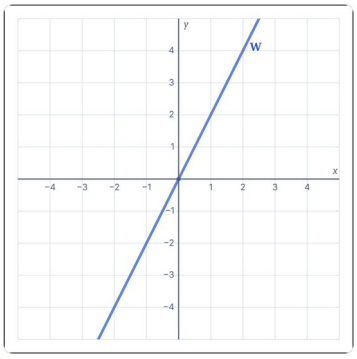
İddia: W = {(x, y) : xy ≥ 0} alt uzay mı?



1. **Küme: 1. ve 3. bölgeler (eksenler dahil):**
2. **1) Sıfır vektörü:** Sıfır var, ama bu tek başına yetmez.
   $0 \cdot 0 = 0 \geq 0$
3. **2) Skalerle çarpma: genel ispat:**
   $(\lambda x)(\lambda y) = \lambda^2 \cdot xy$
   $\lambda^2 \geq 0$ ve $xy \geq 0 \Rightarrow \lambda^2 xy \geq 0$
   Gerekçe: Bir gerçek sayının karesi negatif olamaz; işaret kuralı.
4. **3) Toplama: karşı örnek:** u ve v kümede (biri 1., diğeri 3. bölgede), ama toplamları 2. bölgeye düşüyor.
   $u + v = (-2, 1)$
   $(-2) \cdot (1) = -2 < 0$
5. **Sonuç:** Sıfır var, skalerle kapalı, ama toplamaya kapalı değil ⇒ alt uzay **değil**. (İki doğrunun/bölgenin birleşimi genelde alt uzay değildir.)

Teorem: tek adımda alt uzay testi

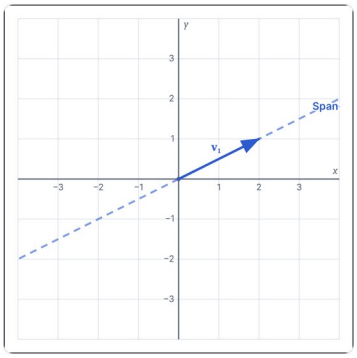
İddia: $W \neq \emptyset$ olsun. W alt uzaydır $\Leftrightarrow$ her $u, v \in W$ ve her α, β için $\alpha u + \beta v \in W$.



1. **Neden bu teorem?** İki kapalılık koşulunu tek bir koşulda birleştirir. Ispat iki yönlüdür ($\Leftrightarrow$): iki ayrı iddia ispatlanmalı.
2. **($\Rightarrow$) W alt uzay ise:**
   $\alpha u \in W,\ \beta v \in W$ (skalerle kapalılık)
   $\alpha u + \beta v \in W$ (toplamaya kapalılık)
3. **($\Leftarrow$) Koşul sağlanıyorsa:**
   $\alpha = \beta = 1$: $u + v \in W$
   $\beta = 0$: $\alpha u + 0 \cdot v = \alpha u + 0 = \alpha u \in W$
   $\alpha = \beta = 0$: $0 \in W$
   Gerekçe: $0 \cdot v = 0$ (Teorem 4.1.1) ve V2 kullanıldı.
4. **Uygulama: y = 2x:**
   $\alpha(r, 2r) + \beta(s, 2s) = (\alpha r + \beta s, 2(\alpha r + \beta s)) \in W$
5. **Sonuç:** İki yön de ispatlandı.

Teorem 4.1.3: Span(S) bir alt uzaydır

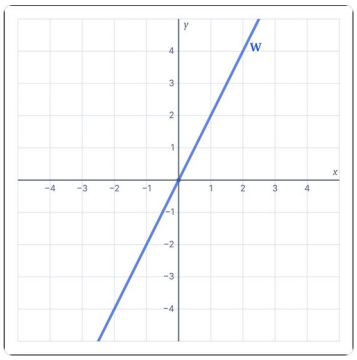
İddia: $S = \{v_1, \ldots, v_k\} \subset V$ ise $\text{Span}(S) = \{c_1v_1 + \ldots + c_kv_k\}$, V'nin bir alt uzayıdır.



1. **Tanım:** Grafikte k = 1: $S = \{v_1\}$, span bir doğru. İspat ise herhangi k için yazılıyor.
2. **1) $0 \in$ Span(S):**
   $0 = 0 \cdot v_1 + \ldots + 0 \cdot v_k$
   Gerekçe: Teorem 4.1.1 ve V2.
3. **2) Toplamaya kapalı:**
   $x = \Sigma\, a_i v_i,\quad y = \Sigma\, b_i v_i$
   $x + y = \Sigma\, (a_i + b_i)\, v_i \in \text{Span}(S)$
   Gerekçe: V1, V4 ile terimleri yeniden grupla; V6 ile $a_iv_i + b_iv_i = (a_i + b_i)v_i$.
4. **3) Skalerle çarpmaya kapalı:**
   $\lambda x = \Sigma\, (\lambda a_i)\, v_i \in \text{Span}(S)$
   Gerekçe: V5 ve V7.
5. **Sonuç:** Span(S) alt uzaydır. Üstelik S'yi içeren **en küçük** alt uzaydır: S'yi içeren her alt uzay kapalılık gereği bütün $c_1v_1 + \ldots + c_kv_k$ birleşimlerini de içermek zorundadır.

Mühendislik: Ax = 0 alt uzaydır, Ax = b değildir

İddia: Homojen doğrusal sistemlerin çözüm kümesi her zaman bir alt uzaydır.



1. **y = 2x aslında bir denklem sistemi:** $2x - y = 0$ denkleminin çözümleri tam olarak $y = 2x$ doğrusudur. Matris dilinde $A = [2\ \ -1]$, $x = (x, y)^T$.
   $Ax = 0,\quad A = [2\ \ -1]$
2. **Toplama:**
   $Ax_1 = 0,\ Ax_2 = 0 \Rightarrow A(x_1 + x_2) = Ax_1 + Ax_2 = 0 + 0 = 0$
   Gerekçe: Matris çarpımı doğrusaldır: $A(x + y) = Ax + Ay$.
3. **Skalerle çarpma:**
   $A(\lambda x_1) = \lambda(Ax_1) = \lambda \cdot 0 = 0$
4. **Homojen olmayan: Ax = b, b ≠ 0:** $2x - y = -1$, yani $y = 2x + 1$ (kesikli doğru). 0 bir çözüm değildir $\Rightarrow$ alt uzay değil.
   $A \cdot 0 = 0 \neq b$
5. **Ama yapı kaybolmaz: x = p + h:** Ax = b'nin her çözümü bir **özel çözüm** p ile bir **homojen çözüm** h'nin toplamıdır. Çözüm kümesi, alt uzayın p kadar kaydırılmış hâlidir. Devre analizi ve titreşimlerde "özel + homojen çözüm" tam olarak budur.
   $A(p + h) = Ap + Ah = b + 0 = b$
6. **Aynı ispat, diferansiyel denklemler için: süperpozisyon:** $y'' + y = 0$ denkleminin çözümleri de bir alt uzaydır (fonksiyon uzayında). İki çözümün her doğrusal birleşimi yine çözümdür: mühendislikteki **süperpozisyon ilkesi**. Doğrusal Germe sekmesindeki fazör laboratuvarında dene.
   $(\alpha y_1 + \beta y_2)'' + (\alpha y_1 + \beta y_2) = \alpha(y_1'' + y_1) + \beta(y_2'' + y_2) = \alpha \cdot 0 + \beta \cdot 0 = 0$
   Gerekçe: Türev doğrusaldır: $(\alpha f + \beta g)' = \alpha f' + \beta g'$.
7. **Özet:** Homojen $\Rightarrow$ alt uzay (süperpozisyon geçerli). Homojen değil $\Rightarrow$ alt uzay değil,

ama “özel çözüm + alt uzay”.

4.1.2 Doğrusal Germe

Tanım 4.1.4. $c_1v_1 + c_2v_2 + \ldots + c_kv_k$ biçimindeki vektöre $v_1, \ldots, v_k$'nin **doğrusal birleşimi** denir. S'nin bütün (sonlu) doğrusal birleşimlerinin kümesi **Span(S)**'dir; Span(∅) = {0}. Span(S) = V ise S, V'yi **gerer**.

Rank = pivot sayısı. Vektörler $\mathbb{R}^m$'de ise: rank = m ⇔ $\mathbb{R}^m$'nin tamamı gerilir. Değilse span'i tanımlayan homojen denklemler (koşullar) çıkar.

"Bir hedefe ulaşılır mı?" (üyelik) ile "her hedefe ulaşılır mı?" (germe) farklı sorulardır: ilki bir Ax = b sistemi, ikincisi rank sorusudur.

$\mathbb{R}^2$'de üç durum

- **rank 2**: det ≠ 0, span = $\mathbb{R}^2$ (düzlem)
- **rank 1**: orijinden geçen doğru
- **rank 0**: yalnızca orijin

Sayılarla tarama (noktalar) yalnızca seçili katsayı aralığıdır; gerçek span sonsuzdur. Hükmü ispat verir.

core/arithmetic.py (satır 163–178)

```
def columns_as_matrix(
    vectors: Sequence[Sequence[Fraction]],
) -> List[List[Fraction]]:
    boyut = len(vectors[0])
    return [[vectors[j][i] for j in range(len(vectors))] for i in range(boyut)]
```

core/vecexpr.py (satır 91–94)

```
def _det(u: Deger, v: Deger) -> Fraction:
    if not (is_vector(u) and is_vector(v) and len(u) == 2 and len(v) == 2):
    return u[0] * v[1] - u[1] * v[0]
```

columns_as_matrix: vektörleri matrisin sütunları olarak yerleştirir. $c_1v_1 + \ldots + c_kv_k = w$ denklemi böylece Ax = w sistemine dönüşür; span/üyelik sorularının hepsi bu kurulumla başlar. _det: $\mathbb{R}^2$'de iki vektörün determinantı $u_1v_2 - u_2v_1$. det ≠ 0 ise iki vektör $\mathbb{R}^2$'yi gerer (Cramer dersi), det = 0 ise aynı doğrudadır.

Süperpozisyon ve fazörler

$y'' + y = 0$ denkleminin (ör. sürtünmesiz yay, kütle, LC devresi) bütün çözümleri Span{cos t, sin t} 'dir. Her çözüm $y(t) = a\cdot\cos t + b\cdot\sin t$, yani tek bir katsayı vektörü (a, b) ∈ $\mathbb{R}^2$ ile tarif edilir. Elektrikte buna fazör denir.

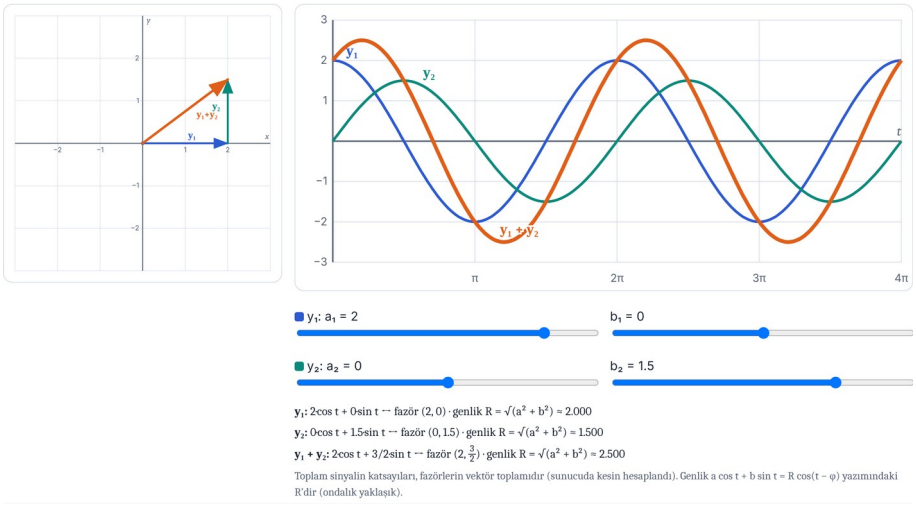
İki sinyali toplamak = fazörlerini (vektörleri) toplamak. Aşağıda solda $\mathbb{R}^2$'deki fazörler, sağda zaman grafiği aynı anda değişir.

$c_1\cdot\cos t + c_2\cdot\sin t = 0$ her t için sağlansın (fonksiyon uzayında sıfır vektörü = sıfır fonksiyonu).

t = 0 koy: $c_1\cdot 1 + c_2\cdot 0 = 0 \Rightarrow c_1 = 0$. t = π/2 koy: $c_1\cdot 0 + c_2\cdot 1 = 0 \Rightarrow c_2 = 0$.

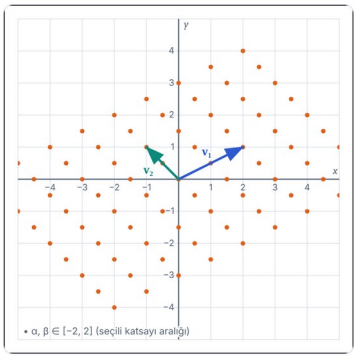
Yalnızca aşikâr çözüm ⇒ {cos t, sin t} bağımsız; ve çözüm uzayını gerdiği için bir baz dır: çözüm uzayının boyutu 2'dir. (a, b) katsayıları bu bazdaki koordinatlardır.

Dikkat: tersi yönde "birkaç t değerinde sıfır çıktı" demek bağımlılık ispatı değildir; bağımlılık için eşitlik HER t'de sağlanmalıdır.



Doğrusal birleşim adım adım: $w = \alpha v_1 + \beta v_2$

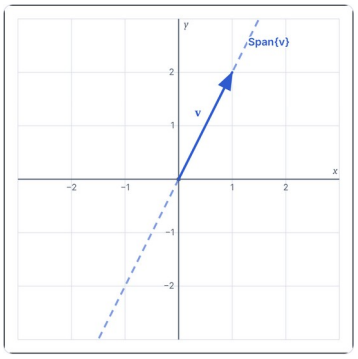
İddia: Doğrusal birleşim = ölçekle, sonra uç uca ekle.



1. **Malzemeler: iki yön:**
2. **α ile ölçekle:**
   $\alpha v_1 = 3/2 \cdot (2, 1) = (3, 3/2)$
3. **β ile ölçekle ve αv_1'in ucuna taşı:**
   $\beta v_2 = (-2, 2)$
4. **Sonuç oku:** α ve β, w'ye "v_1 yönünde ne kadar, v_2 yönünde ne kadar git" tarifidir.
   $w = \alpha v_1 + \beta v_2 = (1, 7/2)$
5. **Tüm birleşimler (seçili katsayı aralığı):** α, β ∈ [−2, 2] aralığında 0.5 adımla tarandı. Noktalar bir **örnek**tir; gerçek span tüm α, β ∈ ℝ için oluşan sonsuz kümedir.
6. **Doğrusal birleşim ≠ span:** Tek bir w bir **doğrusal birleşim**dir. Oluşturabileceğin **bütün** w'ler span'dir. Span'in ne olduğunu (doğru mu, düzlem mi) bir sonraki dersler ispatlıyor.

Tek vektörün span'i: orijinden geçen doğru

İddia: v ≠ 0 ise Span{v} = {tv : t ∈ ℝ} orijinden geçen bir doğrudur.

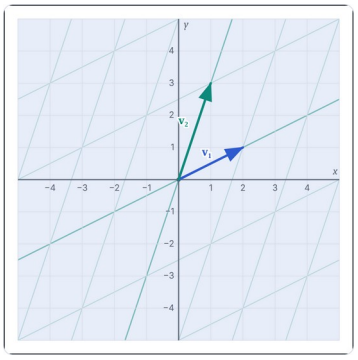


1. **Tanım:**
   $\text{Span}\{v\} = \{tv : t \in \mathbb{R}\}$
2. **Katları tara (seçili aralık $t \in [-3, 3]$):**
3. **Genel: hepsi tek bir doğru üzerinde:** Her tv noktası $qx - py = 0$ denklemini sağlar: bu, orijinden geçen bir doğrunun denklemidir (v ≠ 0 olduğu için p, q'dan biri sıfır değil). Tersine, doğrunun her noktası bir tv'dir.
   $v = (p, q),\ \ tv = (tp, tq)$
   $q \cdot (tp) - p \cdot (tq) = 0$
   Gerekçe: $\mathbb{R}$'de çarpma değişmeli: $q \cdot t \cdot p = p \cdot t \cdot q$.
4. **Özel durum: v = 0:** $t \cdot 0 = 0$ olduğundan Span{0} = {0}: yalnızca orijin (rank 0). Ayrıca tanım gereği Span(∅) = {0}.
5. **Sonuç:**
   $\text{rank } 1 \ \Leftrightarrow$ span bir doğru

---

det ≠ 0 ise iki vektör $\mathbb{R}^2$'yi gerer (Cramer ile ispat)

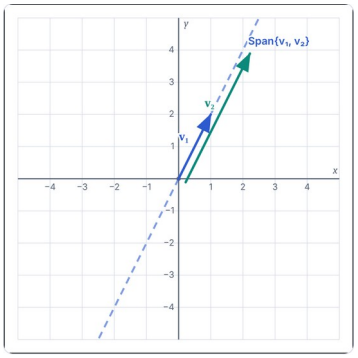
İddia: $v_1 = (p_1, p_2)$, $v_2 = (q_1, q_2)$ ve $D = p_1 q_2 - p_2 q_1 \neq 0$ ise HER $w \in \mathbb{R}^2$ bir $\alpha v_1 + \beta v_2$'dir.



1. **Hedef keyfi:** "Gerer" demek: **her** hedefe ulaşılır. Bu yüzden w'yi keyfi alıp α, β için genel bir formül bulacağız.
   $w = (x, y)$ keyfi
2. **Denklem sistemi:**
   $\alpha v_1 + \beta v_2 = w \quad \Leftrightarrow \quad p_1\alpha + q_1\beta = x, \quad p_2\alpha + q_2\beta = y$
   Gerekçe: Bileşenleri eşitle.
3. **Çöz (Cramer kuralı):**
   $\alpha = (x \cdot q_2 - y \cdot q_1) / D, \quad \beta = (p_1 \cdot y - p_2 \cdot x) / D$
   Gerekçe: $D \neq 0$ olduğu için bölme serbest. 1. denklemi q_2, 2.'yi q_1 ile çarpıp çıkar ⇒ $D\alpha = xq_2 - yq_1$.
4. **Doğrula (yerine koy):**
   $p_1\alpha + q_1\beta = [p_1(xq_2 - yq_1) + q_1(p_1y - p_2x)] / D = x(p_1q_2 - p_2q_1) / D = x$
   $p_2\alpha + q_2\beta = y$ (aynı hesap)
   Gerekçe: Bulunan değer gerçekten çözüm mü? Her zaman kontrol et.
5. **Geometri: D = paralelkenar alanı:** $|D|$, v_1 ve v_2'nin oluşturduğu paralelkenarın alanıdır. $D \neq 0 \Leftrightarrow$ paralelkenar çökmemiş $\Leftrightarrow$ vektörler aynı doğruda değil. Eğik ızgara, v_1 ve v_2 ile kurulan "koordinat kağıdı"dır: düzlemi tamamen kaplar.
6. **Bu hedef için formül ne diyor?**
   $\alpha = 2, \quad \beta = -1$
7. **Sonuç:** Formül **her** (x, y) için çalıştı ⇒ $\text{Span}\{v_1, v_2\} = \mathbb{R}^2$. Örnekle değil, formülle ispatladık; w'yi değiştir ya da 'e bas.
   $D \neq 0 \Rightarrow \text{Span}\{v_1, v_2\} = \mathbb{R}^2$

Çakışık iki vektör $\mathbb{R}^2$'yi germez

İddia: $v_2 = k \cdot v_1$ ise $\text{Span}\{v_1, v_2\}$ yalnızca bir doğrudur.



1. **İki vektör, tek yön:**
   $v_2 = 2 \cdot v_1 = (2, 4), \quad D = \det(v_1, v_2) = 0$
2. **Her birleşim v_1'in bir katı:** 2 boyutlu bir (α, β) tablosu taradık ama bütün noktalar tek bir doğruya çöktü!
   $\alpha v_1 + \beta v_2 = \alpha v_1 + \beta(kv_1) = (\alpha + k\beta)v_1$
   Gerekçe: V7 ve V6.
3. **Ulaşılamayan tek bir hedef yeter:** $e = (1, 0)$ doğru üzerinde değil ($\det(v_1, e) = -2 \neq 0$), yani hiçbir t için $tv_1 = e$ olamaz. "Gerer" iddiası "her w için" dediğinden tek bir karşı örnek onu çürütür.
4. **Sonuç:** $\text{Span}\{v_1, v_2\} = \text{Span}\{v_1\}$: bir doğru, rank 1. Dikkat: iki vektör olması yetmez; ölçüt **rank** (veya $D \neq 0$).

4.2 Bağımsızlık ve Baz

Bağımsızlık. $\{v_1, \ldots, v_k\}$ kümesi için $c_1v_1 + \ldots + c_kv_k = 0$ denkleminin **tek** çözümü $c_1 = \ldots = c_k = 0$ ise küme **doğrusal bağımsızdır**; sıfır olmayan bir çözüm varsa **bağımlıdır**.

Ölçüt. Vektörleri bir matrisin sütunlarına yaz: her sütunda pivot varsa (rank = k) bağımsız; serbest sütun varsa bağımlı. $\mathbb{R}^m$'de m'den fazla vektör daima bağımlıdır (rank ≤ m < k). Eşit sayıda vektör de bağımlı olabilir.

Baz. B, V'nin bazı ⇔ B bağımsız ve Span(B) = V. Her vektör baz cinsinden **tek bir** biçimde yazılır; bu katsayılar koordinatlardır. Bazdaki vektör sayısı **boyut**tur: dim $\mathbb{R}^n$ = n, dim P_n = n + 1, dim $M_{2\times2}$ = 4.

Mühendislik gözüyle

- **Statik denge:** ΣF = 0 olan kuvvetler kapalı bir çokgen oluşturur ⇒ doğrusal bağımlıdır.
- **Koordinat sistemi seçimi** (eğik düzlem, robot eklemleri) = baz seçimi.
- **Diferansiyel denklemler:** {cos t, sin t} çözüm uzayının bazıdır (bkz. fazör laboratuvarı).

$\mathbb{R}^n$'de kendi vektörlerinle bağımsızlık testi ve baz seçimi: "Kendi Vektörlerin" sekmesi.

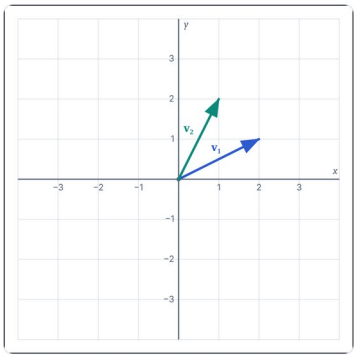
`core/analysis.py (satır 413–432)`

```python
def null_space(vectors: Sequence[Sequence[Fraction]]) -> Dict:
    A = arithmetic.columns_as_matrix(vectors)
    elim = Eliminator(A, [Fraction(0)] * len(A)).rref()
    pivot_row_of = {p["column"]: p["row"] for p in elim.pivots}
    temel = []
    for f in elim.free_columns():
        d = [Fraction(0)] * elim.n
        d[f] = Fraction(1)
        for c in elim.pivot_columns():
            d[c] = -elim.A[pivot_row_of[c]][f]
        temel.append(_integerize(d))
    return {"elim": elim, "basis": temel}
```

$c_1v_1 + \ldots + c_kv_k = 0$ homojen sistemini RREF'e getirir. Her serbest (pivotsuz) sütun için o katsayıyı 1, diğer serbest katsayıları 0 alıp pivot katsayılarını geri okur → sıfır olmayan bir bağıntı. Serbest sütun yoksa liste boş döner: küme bağımsızdır. _integerize bağıntıyı en sade tam sayılı hale getirir.

Doğrusal bağımsızlık: tanım ve $\mathbb{R}^2$'de ispat

İddia: $\{v_1, v_2\}$ bağımsızdır ⇔ $c_1v_1 + c_2v_2 = 0$ denkleminin TEK çözümü $c_1 = c_2 = 0$'dır.



1. **Tanım:** $c_1 = \ldots = c_k = 0$ her zaman bir çözümdür (**aşikâr çözüm**). Bağımsızlık: başka çözüm olmaması. Bağımlılık: sıfır olmayan katsayılarla 0'a ulaşılabilmesi.

$$c_1v_1 + c_2v_2 + \ldots + c_kv_k = 0 \;\Rightarrow\; c_1 = c_2 = \ldots = c_k = 0$$

2. **Denklemi kur:**

$$p_1c_1 + q_1c_2 = 0$$
$$p_2c_1 + q_2c_2 = 0$$

3. **Çöz:**

$$D = p_1q_2 - p_2q_1 = 3 \neq 0$$
$$c_1 = (0\cdot q_2 - 0\cdot q_1)/D = 0, \quad c_2 = (p_1\cdot 0 - p_2\cdot 0)/D = 0$$

Gerekçe: Cramer (Germe sekmesindeki ispat), sağ taraf w = 0 ile.

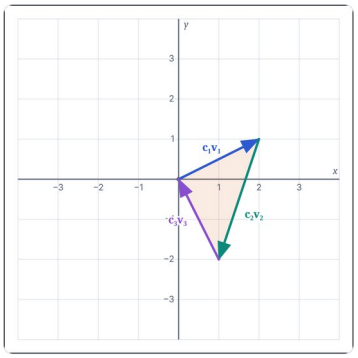
4. **Geometrik anlam:** Orijinden çıkıp "v_1 yönünde, sonra v_2 yönünde" yürüyerek orijine geri dönmenin tek yolu **hiç yürümemek**. Paralelkenar çökmüş değil.

5. **Sonuç:** Genel ölçüt (her boyutta): vektörleri sütun yap; **her sütunda pivot varsa** (rank = vektör sayısı) bağımsızdır. "Sıfır sütun yok" bir ölçüt değildir.

$$D \neq 0 \;\Rightarrow\; \{v_1, v_2\} \text{ bağımsız}$$

$\mathbb{R}^2$'de üç vektör her zaman bağımlıdır: kapanan çokgen

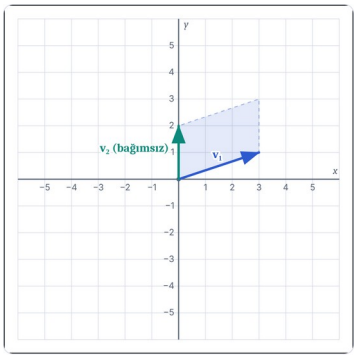
İddia: $\mathbb{R}^2$'deki herhangi üç vektör doğrusal bağımlıdır.



1. **Üç vektör:**
2. **Neden mutlaka bağımlı? (genel ispat):** Dikkat: "fazla vektör ⇒ bağımlı" yalnızca vektör sayısı boyutu aştığında bir teoremdir. Eşit sayıda vektör de bağımlı olabilir (ör. (1, 2) ve (2, 4)); asıl ölçüt rank'tir.
   $c_1v_1 + c_2v_2 + c_3v_3 = 0$: 2 denklem, 3 bilinmeyen
   $\text{rank} \le 2 < 3 \Rightarrow$ en az bir serbest değişken
   $\Rightarrow$ sıfır olmayan bir çözüm VAR
   Gerekçe: Homojen sistemde serbest değişkene 1 verip pivot değişkenleri çözeriz.
3. **Motor çözümü buldu:**
   $(c_1, c_2, c_3) = (1, -1, 1)$
   Gerekçe: Satır indirgeme (RREF) + serbest değişken = 1, kesin kesirli aritmetik.
4. **Yürü: c_1v_1:**
5. **Uç uca: + c_2v_2:**
6. **Uç uca: + c_3v_3 ... ve orijine döndük!**
   $c_1v_1 + c_2v_2 + c_3v_3 = (0, 0)$
7. **Sonuç:** Sıfır olmayan katsayılarla 0'a döndük ⇒ bağımlı. Katsayısı sıfır olmayan her vektör, diğerlerinin doğrusal birleşimidir (fazladır): onu atsak span değişmez. Fizikte bunun adı **kapalı kuvvet çokgeni**dir (bkz. Kuvvet dengesi).

İki vektör: bağımlı ⇔ aynı doğruda ⇔ det = 0

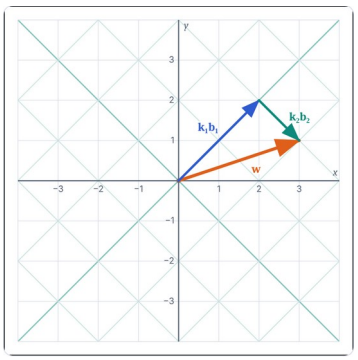
İddia: $v_1 = (p_1, p_2)$, $v_2 = (q_1, q_2)$ için: $\{v_1, v_2\}$ bağımlı ⇔ $D = p_1q_2 - p_2q_1 = 0$



1. **İzle: v_2, v_1'in doğrusuna yaklaşıyor:**
2. **Alan küçülüyor...:**
3. **... iyice küçülüyor ...:**
4. **Çöktü: $v_2 = (2/3)v_1$:**
   $(2/3)\cdot v_1 - 1\cdot v_2 = 0$ (sıfır olmayan katsayılar)
5. **İspat (⇒): bağımlı ise D = 0:**
   $c_1v_1 + c_2v_2 = 0$, örneğin $c_2 \neq 0 \Rightarrow v_2 = k\cdot v_1$ $(k = -c_1/c_2)$
   $D = p_1(kp_2) - p_2(kp_1) = 0$
   Gerekçe: $c_1 \neq 0$ durumu simetrik.
6. **İspat (⇐): D = 0 ise bağımlı:** Şu iki özdeşliği bileşen bileşen kontrol et:
   $q_2\cdot v_1 - p_2\cdot v_2 = (D, 0)$
   $q_1\cdot v_1 - p_1\cdot v_2 = (0, -D)$
   $D = 0 \Rightarrow$ ikisi de 0. $v_2 \neq 0$ ise katsayılardan biri $\neq 0 \Rightarrow$ bağımlı.
   $v_2 = 0$ ise $0\cdot v_1 + 1\cdot v_2 = 0 \Rightarrow$ yine bağımlı.
   Gerekçe: Durum ayırma: hiçbir olasılık atlanmamalı.
7. **Sonuç:** $\mathbb{R}^2$'de iki vektör için: bağımlı ⇔ aynı doğruda ⇔ paralelkenar alanı 0 ⇔ D = 0. $\mathbb{R}^n$'de genel ölçüt: rank < vektör sayısı.

Baz: bağımsız + geren küme = koordinat sistemi

İddia: $B = \{b_1, b_2\}$ $\mathbb{R}^2$'nin bazıysa her w TEK bir biçimde $w = k_1b_1 + k_2b_2$ yazılır.

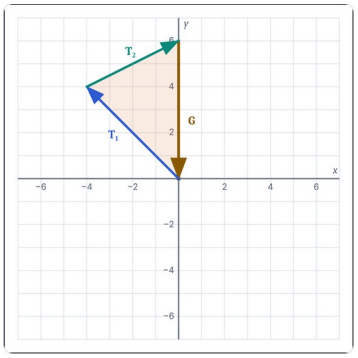


1. **Tanım:** B, V'nin **bazı** ⇔ (i) B bağımsız, (ii) Span(B) = V. Bir bazdaki vektör sayısı uzayın **boyutu**dur ($\mathbb{R}^2$ için 2, P_n için n + 1).
2. **B gerçekten baz mı?**
   $D = \det(b_1, b_2) = 2 \neq 0$
   Gerekçe: $D \neq 0 \Rightarrow$ bağımsız (B_det) ve $\mathbb{R}^2$'yi gerer (Cramer). İki koşul birden.
3. **w'nin B-koordinatlarını bul:**
   $k_1 = \det(w, b_2)/D = 2$, $k_2 = \det(b_1, w)/D = -1$
   $[w]_B = (2, -1)$
4. **Eğik ızgarada yürü: k_1b_1:**
5. **... sonra k_2b_2:**
6. **Koordinatlar neden TEK? (genel ispat):**
   $w = \Sigma\, a_ib_i = \Sigma\, c_ib_i$
   $\Rightarrow \Sigma\, (a_i - c_i)\, b_i = 0$
   $\Rightarrow a_i - c_i = 0$ (bağımsızlık) $\Rightarrow a_i = c_i$
   Gerekçe: Germe ⇒ en az bir temsil var; bağımsızlık ⇒ en fazla bir temsil var.

7. Sonuç: Baz = koordinat sistemi. Standart bazda w = (3, 1); B bazında $[w]_B = (2, -1)$. Aynı vektör, farklı "dil". Mühendislikte: eğik düzlemde "eğim boyunca / eğime dik" bileşenler, robot eklem eksenleri, kristal kafesleri.

Mühendislik: iki kabloyla asılı yük (statik denge)

İddia: Denge koşulu ΣF = 0, bir doğrusal birleşim / baz problemidir.



1. Problem: Bir kütle iki kabloyla asılı. Kablo yönleri d_1, d_2; ağırlık G = (0, −6) (N). Kablo kuvvetleri $T_1 = k_1 d_1$ ve $T_2 = k_2 d_2$ ($k \geq 0$: kablo yalnızca çeker). Denge (Newton'un 1. yasası):

$$T_1 + T_2 + G = 0$$

2. Doğrusal cebir diline çevir: Aranan (k_1, k_2), −G'nin $\{d_1, d_2\}$ bazındaki **koordinatlarıdır**.

$$k_1 d_1 + k_2 d_2 = -G = (0, 6)$$

3. Çöz (Cramer):

$$D = \det(d_1, d_2) = -3 \neq 0$$
$$k_1 = 4, \quad k_2 = 2$$
$$T_1 = (-4, 4), \quad T_2 = (4, 2)$$

Gerekçe: d_1, d_2 bağımsız ⇒ $\{d_1, d_2\}$ bir baz ⇒ çözüm VAR ve TEK (koordinatların tekliği).

4. Kuvvet çokgeni kapanıyor: T_1, T_2 ve G'yi uç uca ekle: orijine dönüyoruz. Kapalı kuvvet çokgeni ⇔ $\{T_1, T_2, G\}$ katsayıları (1, 1, 1) olan bir **doğrusal bağımlılık** sağlıyor.

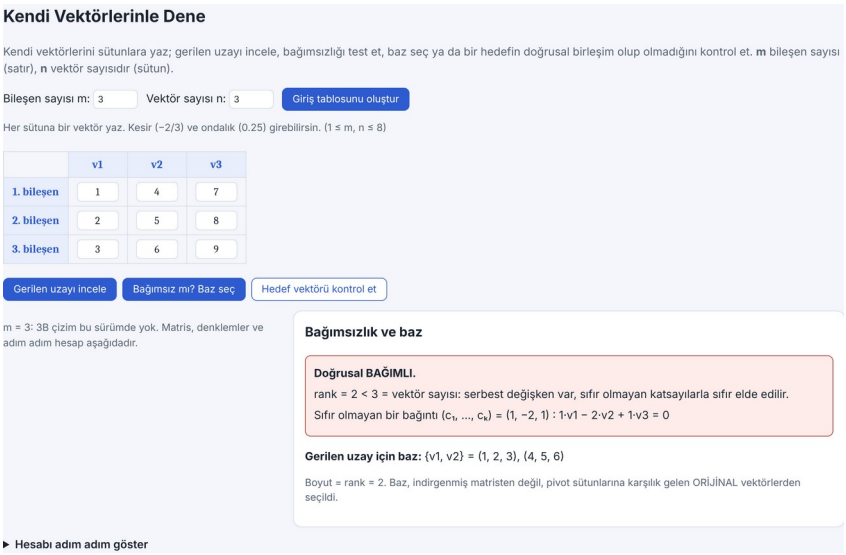
$$T_1 + T_2 + G = (0, 0)$$

5. Fiziksel kontrol: Matematik $k_1 = 4$, $k_2 = 2$ verdi. Kablolar yalnızca çekebildiği için gerçek bir denge için $k_1 > 0$ ve $k_2 > 0$ olmalı. Negatif çıkarsa bu kablo düzeni yükü taşıyamaz (kablonun itmesi gerekirdi). Matematik sonucu fiziksel kısıtla birlikte yorumlanır.

6. Özet: Statik denge = "−G'yi kablo yönleri bazında yaz". D = 0 (paralel kablolar) ⇒ yönler baz değil ⇒ genelde denge yok. Kafes sistemlerinde aynı fikir, $\mathbb{R}^{2n}$'de büyük bir Ax = b sistemine dönüşür.

Kendi vektörlerimle deneme

Uygulamada vektörleri bir tabloya sütun sütun yazıp bağımsız olup olmadıklarını test edebiliyoruz. Aşağıda (1, 2, 3), (4, 5, 6), (7, 8, 9) vektörlerini girdim. Program bunların bağımlı olduğunu ve baz olarak ilk iki vektörün seçilebileceğini buldu.



Sorular

Sorularda vektörleri matrisin sütunlarına yazıp satır işlemleriyle indirgedim. Her satır işlemi ayrı bir adım olarak gösterildi. Bütün sorular core/elimination.py dosyasındaki aynı indirgeme koduyla çözülüyor:

core/elimination.py (satır 189–219)

```
def ref(self) -> "Eliminator":
    pivot_row = 0
    for col in range(self.n):
        if pivot_row >= self.m:
            break
        sec = self._pick_pivot(pivot_row, col)
        if sec is None:
            continue
        if sec != pivot_row:
            self.swap(
                pivot_row,
                sec,
                f"{col + 1}. sütunda pivot için satırları değiştiriyoruz.",
                pivot={"row": pivot_row, "column": col},
            )
        p = self.A[pivot_row][col]
        for r in range(pivot_row + 1, self.m):
            e = self.A[r][col]
            if e == 0:
                continue
            factor = e / p
            self.combine(
                r,
                pivot_row,
                factor,
                f"{col + 1}. sütundaki {format_fraction(e)} değerini sıfırlıyoruz.",
                pivot={"row": pivot_row, "column": col},
            )
        self.pivots.append({"row": pivot_row, "column": col})
        pivot_row += 1
    return self
```

core/elimination.py (satır 221–244)

```
def rref(self) -> "Eliminator":
    self.ref()
    for p in reversed(self.pivots):
        r, c = p["row"], p["column"]
        pivot_val = self.A[r][c]
        if pivot_val != 1:
            self.scale(
                r,
                Fraction(1) / pivot_val,
                f"{c + 1}. sütundaki pivotu 1 yapıyoruz.",
                pivot=p,
            )
        for ust in range(r):
            e = self.A[ust][c]
            if e == 0:
                continue
            self.combine(
                ust,
```

```
                r,
                e,
                f"{c + 1}. sütunda pivotun üstünü sıfırlıyoruz.",
                pivot=p,
            )
    return self
```

ref() soldan sağa her sütunda bir pivot arıyor, gerekirse satırları değiştiriyor ve pivotun altını sıfırlıyor, böylece basamak biçimi (REF) elde ediliyor. rref() bundan sonra pivotları 1 yapıp üstlerini de sıfırlıyor (RREF). Pivot sayısı rank oluyor, pivotu olmayan sütunlar serbest değişkenlere karşılık geliyor.

Soru 17: Doğrusal birleşim

Soru: (4, -2, 4) vektörü (1, 0, 1), (3, -4, 1), (4, -6, -2) vektörlerinin doğrusal birleşimi olarak yazılabilir mi?

İstenen: Hedef vektörü verilen üç vektörün doğrusal birleşimi yapan katsayılar var mı?

Adım 1: Başlangıç matrisi

1	3	4	4
0	−4	−6	−2
1	1	−2	4

Adım 2: R3 ← R3 − R1

1. sütundaki 1 değerini sıfırlıyoruz.

1	3	4	4
0	−4	−6	−2
0	−2	−6	0

Adım 3: R3 ← R3 − 1/2R2

2. sütundaki -2 değerini sıfırlıyoruz.

1	3	4	4
0	**−4**	−6	−2
0	0	−3	1

Adım 4: R3 ← -1/3·R3

3. sütundaki pivotu 1 yapıyoruz.

1	3	4	4
0	−4	−6	−2
0	0	**1**	$-\frac{1}{3}$

Adım 5: R1 ← R1 − 4R3

3. sütunda pivotun üstünü sıfırlıyoruz.

1	3	0	$\frac{16}{3}$
0	−4	−6	−2
0	0	**1**	$-\frac{1}{3}$

Adım 6: R2 ← R2 + 6R3

3. sütunda pivotun üstünü sıfırlıyoruz.

1	3	0	$\frac{16}{3}$
0	−4	0	−4
0	0	**1**	$-\frac{1}{3}$

Adım 7: R2 ← -1/4·R2

2. sütundaki pivotu 1 yapıyoruz.

1	3	0	$\frac{16}{3}$
0	**1**	0	1
0	0	1	$-\frac{1}{3}$

Adım 8: R1 ← R1 − 3R2

2. sütunda pivotun üstünü sıfırlıyoruz.

1	0	0	$\frac{7}{3}$
0	**1**	0	1
0	0	1	$-\frac{1}{3}$

Kırmızı hücre pivotu, renkli satır o adımda değişen satırı gösteriyor. Dikey çizginin sağı sağ taraftır.

İndirgenmiş (RREF) matrisin her satırı bir denklemdir; çözüm doğrudan okunur:

> $\lambda_1 = 7/3$
> $\lambda_2 = 1$
> $\lambda_3 = -1/3$

Bulunan katsayılar gerçekten hedefi veriyor mu? (Kesin kesirlerle yeniden hesaplandı.)

> $7/3 \cdot (1, 0, 1) + (3, -4, 1) - 1/3 \cdot (4, -6, -2) = (4, -2, 4)$

Her katsayı sütununda pivot var (rank = 3 = vektör sayısı) ⇒ v_1, v_2, v_3 doğrusal bağımsız ⇒ temsil tektir; başka katsayı üçlüsü yoktur. Üstelik rank = 3 = boyut: bu 3 vektör $\mathbb{R}^3$'ün bir bazıdır. Yani sadece bu hedef değil, $\mathbb{R}^3$'teki her vektör tek biçimde yazılabilirdi. Bulunan sayılar hedefin bu bazdaki koordinatlarıdır: [w]_B = (7/3, 1, −1/3).

▼ Sonucu göster

Evet. Katsayılar (tek çözüm): $(\frac{7}{3}, 1, -\frac{1}{3})$

rank = 3 · 3 vektör · ortam uzayı $\mathbb{R}^3$

core/analysis.py (satır 21-93)

```
def solve_combination(
```

```
    vectors: Sequence[Sequence[Fraction]], target: Sequence[Fraction]
) -> Dict:
    m = len(target)
    A = arithmetic.columns_as_matrix(vectors)
    rhs = [Fraction(t) for t in target]
    elim = Eliminator(A, rhs).rref()
    k = len(vectors)
    tutarsiz = False
    for r in range(elim.m):
        if all(elim.A[r][c] == 0 for c in range(elim.n)) and elim.rhs[r] != 0:
            tutarsiz = True
            break
    sonuc: Dict[str, object] = { ... }
    if tutarsiz:
        sonuc["status"] = "inconsistent"
        sonuc["member"] = False
        return sonuc
    pivot_cols = elim.pivot_columns()
    free_cols = elim.free_columns()
    pivot_row_of = {p["column"]: p["row"] for p in elim.pivots}
    # Özel çözüm (serbest değişkenler = 0)
    ozel = [Fraction(0)] * k
    for c in pivot_cols:
        ozel[c] = elim.rhs[pivot_row_of[c]]
    sonuc["member"] = True
    if not free_cols:
        sonuc["status"] = "unique"
        sonuc["coefficients"] = format_vector(ozel)
    else:
        sonuc["status"] = "infinite"
        sonuc["particular"] = format_vector(ozel)
        yonler = []
        for f in free_cols:
            d = [Fraction(0)] * k
            d[f] = Fraction(1)
            for c in pivot_cols:
                d[c] = -elim.A[pivot_row_of[c]][f]
            yonler.append(format_vector(d))
        sonuc["free_columns"] = free_cols
        sonuc["directions"] = yonler
        sonuc["note"] = (
            "Serbest katsayı(lar) sıfır olmak zorunda değildir; tek bir temsil "
            "göstermek için seçilmiştir."
        )
    return sonuc
```

Vektörleri sütun yapıp (columns_as_matrix) hedefi sağ taraf olarak ekler ve Eliminator(...).rref() ile RREF'e indirir. Tutarsızlık kontrolü: katsayıları hep 0 olan ama sağ tarafı 0 olmayan bir satır varsa (0 = sayı) hedef span'de değildir. Serbest sütun yoksa tek çözüm ("unique"): katsayılar pivot satırlarının sağ tarafından okunur. Bu soruda $\lambda_1 = 7/3$, $\lambda_2 = 1$, $\lambda_3 = -1/3$. Serbest sütun varsa sonsuz temsil ("infinite"): serbest katsayılar 0 alınarak bir özel çözüm verilir.

Soru 20: Homojen sistem

Soru: Span{(1, -1, 0), (0, 1, 1), (3, -5, -2)} uzayını homojen doğrusal bir sistemin çözüm kümesi olarak ifade ediniz.

İstenen: Bu üç vektörün gerdiği uzayı tanımlayan homojen denklem(ler) nedir?

Adım 1: Başlangıç matrisi

1	0	3	x
−1	1	−5	y
0	1	−2	z

Adım 2: $R2 \leftarrow R2 + R1$

1. sütundaki -1 değerini sıfırlıyoruz.

1	0	3	x
0	1	−2	x + y
0	1	−2	z

Adım 3: $R3 \leftarrow R3 - R2$

2. sütundaki 1 değerini sıfırlıyoruz.

1	0	3	x
0	**1**	−2	x + y
0	0	0	−x − y + z

Kırmızı hücre pivotu, renkli satır o adımda değişen satırı gösteriyor. Dikey çizginin sağı sağ taraftır.

Katsayı kısmı tamamen sıfırlanan satırlarda sol taraf 0'dır; sağ taraf da 0 olmak zorunda. Bu bir çelişki değil, üyelik koşuludur:

> $x + y - z = 0$

Gerilen uzaydaki her vektör koşulu sağlamalı; vektörlerin kendileri ile deneyelim:

> $v_1 = (1, -1, 0) \rightarrow$ koşul(lar) $= 0$
> $v_2 = (0, 1, 1) \rightarrow$ koşul(lar) $= 0$
> $v_3 = (3, -5, -2) \rightarrow$ koşul(lar) $= 0$
> $(1, 0, 0) \rightarrow$ koşul sağlanmaz $\Rightarrow$ span dışında (o yüzden $\mathbb{R}^3$'ü germez)

rank = 2 ⇒ Span, $\mathbb{R}^3$ içinde orijinden geçen bir düzlem. Boyut ile denklem sayısı toplanınca ortam boyutu çıkar: 2 + 1 = 3. Düzlemin normal vektörü n = (1, 1, −1): koşul, "(x, y, z) · n = 0" demektir.

3 vektör var ama rank 2: 1 tanesi diğerlerinin birleşimi (bağımlı). Aynı uzayı daha az vektörle germek mümkün:

> $v_3 = 3 \cdot v_1 - 2 \cdot v_2$

▼ Sonucu göster

Homojen koşul(lar):

$-x - y + z = 0$

rank = 2 · 3 vektör · ortam uzayı $\mathbb{R}^3$

Doğrulama: HA = 0 → evet ; rank(H) = 1 (beklenen 1) → tutarlı ✓

Sonuç $[1\ 1\ -1][x\ y\ z]^T = 0$ aynı denklemin matris yazımıdır; yeni bir işlem değildir.

core/analysis.py (satır 99-142)

```python
def homogeneous_conditions(
    vectors: Sequence[Sequence[Fraction]], symbols: Sequence[str]
) -> Dict:
    m = len(symbols)
    A = arithmetic.columns_as_matrix(vectors)
    rhs = [LinearExpr.symbol(s) for s in symbols]
    elim = Eliminator(A, rhs, symbols=symbols).ref()
    kosullar: List[LinearExpr] = []
    for r in range(elim.m):
        if all(elim.A[r][c] == 0 for c in range(elim.n)):
            ifade = elim.rhs[r]
            if isinstance(ifade, LinearExpr) and not ifade.is_zero():
                kosullar.append(ifade.clear_denominators())
    H = [k.coefficients(symbols) for k in kosullar]
    dogrulama = _verify_conditions(H, A, elim.rank, m)
    return { ... }
```

core/analysis.py (satır 145-165)

```python
def _verify_conditions(H, A, rankA, m) -> Dict:
    HA_sifir = True
    if H:
        HA = arithmetic.matrix_mul(H, A)
        for row in HA:
            for v in row:
                if v != 0:
                    HA_sifir = False
    rankH = 0
```

```
    if H:
        rankH = Eliminator(H).ref().rank
    beklenen = m - rankA
    return { ... }
```

Sağ taraf sayı değil sembol: (x, y, z). LinearExpr sınıfı x − y + … gibi doğrusal ifadeleri kesin kesirlerle taşır. Sadece REF yeterli: sol tarafı tamamen sıfır olan satırların sağ tarafı "= 0" koşulunu verir. Bu soruda −x − y + z = 0 (yani x + y − z = 0). _verify_conditions iki kontrol yapar: H·A = 0 (koşul her vektörde sağlanıyor mu?) ve rank(H) = m − rank(A) (koşul sayısı doğru mu?).

Örnek 4.1.5: Bütün $\mathbb{R}^3$'ü germek

Soru: {(1,2,-1), (-3,1,0), (2,-3,2)} kümesinin $\mathbb{R}^3$'ü gerdiğini gösteriniz.

İstenen: Bu üç vektör $\mathbb{R}^3$'ün tamamını gerer mi (her hedef için çözüm var mı)?

Adım 1: Başlangıç matrisi

1	−3	2	a
2	1	−3	b
−1	0	2	c

Adım 2: $R2 \leftarrow R2 - 2R1$

1. sütundaki 2 değerini sıfırlıyoruz.

1	−3	2	a
0	7	−7	−2a + b
−1	0	2	c

Adım 3: $R3 \leftarrow R3 + R1$

1. sütundaki -1 değerini sıfırlıyoruz.

1	−3	2	a
0	7	−7	−2a + b
0	−3	4	a + c

Adım 4: $R3 \leftarrow R3 + 3/7R2$

2. sütundaki -3 değerini sıfırlıyoruz.

1	−3	2	a
0	**7**	−7	−2a + b
0	0	1	$\frac{1}{7}a + \frac{3}{7}b + c$

Kırmızı hücre pivotu, renkli satır o adımda değişen satırı gösteriyor. Dikey çizginin sağı sağ taraftır.

Her satırda pivot var; tamamen sıfır olan katsayı satırı oluşmadı. Bu yüzden hedef (a, b, c) üzerinde hiçbir koşul yok: her hedef için sistem çözülebilir.

> son sağ taraf: (a, −2a + b, 1/7a + 3/7b + c)

rank = 3 = boyut ⇒ Span = $\mathbb{R}^3$. 3 vektör ve rank 3: aynı zamanda bağımsızlar ⇒ $\mathbb{R}^3$'ün bir bazı.

▼ Sonucu göster

Rank = boyut: bu vektörler uzayın tamamını gerer; ek koşul yoktur.
rank = 3 · 3 vektör · ortam uzayı $\mathbb{R}^3$
Doğrulama: HA = 0 → evet ; rank(H) = 0 (beklenen 0) → tutarlı ✓
Kaynak düzeltmesi: PDF s.117'de son satırın sağ tarafı hatalı basılmıştır; katsayı satırı [0,0,7] için doğru sağ taraf a+3b+7c olur.

core/analysis.py (ilgili satırlar)

```
# core/analysis.py → homogeneous_conditions (Soru 20 ile aynı fonksiyon)
rhs = [LinearExpr.symbol(s) for s in symbols]      # sağ taraf: (a, b, c)
elim = Eliminator(A, rhs, symbols=symbols).ref()  # REF
...
"spans_ambient": elim.rank == m,                # rank = 3 = boyut ⇒ ℝ³ gerilir
```

Soru 20 ile aynı fonksiyon çalışır; fark sonuçtadır. Her satırda pivot çıktığı için sıfır satır oluşmaz ⇒ hiçbir koşul yoktur. rank = 3 = m olduğundan spans_ambient = True: küme $\mathbb{R}^3$'ü gerer. Son satırın sağ tarafı (1/7)a + (3/7)b + c bulunur; 7 ile çarpılınca a + 3b + 7c olur (kitaptaki baskı hatası burada düzeltilmiştir).

Soru 15: Polinomlar da vektördür

Soru: $1 + x + c{\cdot}x^2 + d{\cdot}x^3$ polinomu şu polinomların doğrusal birleşimi ise c ve d'yi bulunuz: $(-1+2x+3x^2+4x^3)$, $(4+x+6x^2+11x^3)$, $(1-x-x^2-x^3)$, $(3+x+5x^2+9x^3)$.

İstenen: Birleşim $1 + x + c{\cdot}x^2 + d{\cdot}x^3$ polinomunu veriyorsa c ve d kaçtır?

Adım 1: Başlangıç matrisi

−1	4	1	3	1
2	1	−1	1	1
3	6	−1	5	c
4	11	−1	9	d

Adım 2: R2 ← R2 + 2R1

1. sütundaki 2 değerini sıfırlıyoruz.

−1	4	1	3	1
0	9	1	7	3
3	6	−1	5	c
4	11	−1	9	d

Adım 3: R3 ← R3 + 3R1

1. sütundaki 3 değerini sıfırlıyoruz.

−1	4	1	3	1
0	9	1	7	3
0	18	2	14	c + 3
4	11	−1	9	d

Adım 4: R4 ← R4 + 4R1

1. sütundaki 4 değerini sıfırlıyoruz.

−1	4	1	3	1
0	9	1	7	3
0	18	2	14	c + 3
0	27	3	21	d + 4

Adım 5: R3 ← R3 − 2R2

2. sütundaki 18 değerini sıfırlıyoruz.

−1	4	1	3	1
0	**9**	1	7	3
0	0	0	0	c − 3
0	27	3	21	d + 4

Adım 6: R4 ← R4 − 3R2

2. sütundaki 27 değerini sıfırlıyoruz.

−1	4	1	3	1
0	**9**	1	7	3
0	0	0	0	c − 3
0	0	0	0	d − 5

Kırmızı hücre pivotu, renkli satır o adımda değişen satırı gösteriyor. Dikey çizginin sağı sağ taraftır.

Eliminasyon sonunda iki sıfır satır kaldı. Sağ tarafları sıfır olmak zorunda; bu da c ve d'yi belirler:

> $c - 3 = 0$
> $d - 5 = 0$
> $c = 3,\ d = 5$

Katsayı vektörlerini geri polinoma çevirelim: Bu temsil TEK değildir (rank < polinom sayısı): serbest katsayılar 0 seçildi; örneğin $2 \cdot p_1 + 3 \cdot p_3$ de aynı polinomu verir.

> $1/3 \cdot p_1 + 1/3 \cdot p_2 = 1 + x + 3x^2 + 5x^3$

Dört polinom P_3 (boyut 4) içinde yalnızca rank = 2 boyutlu bir alt uzay gerer. Bu yüzden hedefin son iki katsayısı serbest değildi: ilk ikisi (1, 1) seçilince c ve d zorunlu olarak belirlendi.

2 polinom diğerlerinin birleşimidir:

> $p_3 = -5/9 \cdot p_1 + 1/9 \cdot p_2$
> $p_4 = 1/9 \cdot p_1 + 7/9 \cdot p_2$

▼ Sonucu göster

Bilinmeyenler:

c = 3

d = 5

Bir temsil (katsayılar): $(\frac{1}{3}, \frac{1}{3}, 0, 0)$

Polinomlar katsayı vektörlerine dönüştürülür. Katsayı sırası $[a_0, a_1, a_2, a_3]$; grafik yerine katsayı tablosu esastır.

core/analysis.py (satır 168-242)

```python
def solve_target_unknowns(
    vectors: Sequence[Sequence[Fraction]], components: Sequence[str]
) -> Dict:
    unknowns: List[str] = []
    rhs: List[LinearExpr] = []
    for parca in components:
        try:
            deger = arithmetic.parse_fraction(parca)
            rhs.append(LinearExpr.constant(deger))
        except SayiHatasi:
            ad = str(parca).strip()
            rhs.append(LinearExpr.symbol(ad))
            if ad not in unknowns:
                unknowns.append(ad)
    m = len(components)
    A = arithmetic.columns_as_matrix(vectors)
    elim = Eliminator(A, rhs, symbols=unknowns).ref()
```

```
    kosullar: List[LinearExpr] = []
    for r in range(elim.m):
        if all(elim.A[r][c] == 0 for c in range(elim.n)):
            ifade = elim.rhs[r]
            if isinstance(ifade, LinearExpr) and not ifade.is_zero():
                kosullar.append(ifade)
    # Koşulları bilinmeyenler için çöz: katsayı·unknown = -sabit
    coeff = [[k.terms.get(s, Fraction(0)) for s in unknowns] for k in kosullar]
    sab = [-k.const for k in kosullar]
    cozum_degerleri: Dict[str, str] = {}
    temsil = None
    if coeff and unknowns:
        coz = Eliminator(coeff, sab).rref()
        tutarsiz = any(
            all(coz.A[r][c] == 0 for c in range(coz.n)) and coz.rhs[r] != 0
            for r in range(coz.m)
        )
        if not tutarsiz and not coz.free_columns():
            prow = {p["column"]: p["row"] for p in coz.pivots}
            degerler = {}
            for c in range(len(unknowns)):
                degerler[unknowns[c]] = coz.rhs[prow[c]]
            cozum_degerleri = {s: format_fraction(v) for s, v in degerler.items()}
            # Bir temsil: bilinen hedefi kurup katsayıları bul.
            hedef = []
            for parca in components:
                try:
                    hedef.append(arithmetic.parse_fraction(parca))
                except SayiHatasi:
                    hedef.append(degerler[str(parca).strip()])
            temsil = solve_combination(vectors, hedef)
    return { ... }
```

Polinomlar katsayı vektörüne çevrilir: $a_0 + a_1x + a_2x^2 + a_3x^3 \to (a_0, a_1, a_2, a_3)$. Hedefin bilinen bileşenleri (1, 1) sabit, bilinmeyenleri (c, d) sembol olarak sağ tarafa yazılır ve REF yapılır. Sol tarafı sıfırlanan satırlar c ve d için doğrusal denklemler verir; bu küçük sistem tekrar Eliminator ile çözülür. Bulunan c, d ile hedef tamamlanır ve solve_combination ile bir katsayı temsili de hesaplanır (kontrol).

Bağımlı mı, bağımsız mı?

Soru: (1, 2, 3), (4, 5, 6), (7, 8, 9) vektörleri $\mathbb{R}^3$'te doğrusal bağımsız mıdır? Bağımlıysa sıfır olmayan bir bağıntı bulunuz.

İstenen: $c_1v_1 + c_2v_2 + c_3v_3 = 0$ denkleminin yalnızca aşikâr çözümü mü var?

Adım 1: Başlangıç matrisi

1	4	7	0
2	5	8	0
3	6	9	0

Adım 2: $R2 \leftarrow R2 - 2R1$

1. sütundaki 2 değerini sıfırlıyoruz.

1	4	7	0
0	−3	−6	0
3	6	9	0

Adım 3: $R3 \leftarrow R3 - 3R1$

1. sütundaki 3 değerini sıfırlıyoruz.

1	4	7	0
0	−3	−6	0
0	−6	−12	0

Adım 4: $R3 \leftarrow R3 - 2R2$

2. sütundaki -6 değerini sıfırlıyoruz.

1	4	7	0
0	**−3**	−6	0
0	0	0	0

Adım 5: $R2 \leftarrow -1/3 \cdot R2$

2. sütundaki pivotu 1 yapıyoruz.

1	4	7	0
0	**1**	2	0
0	0	0	0

Adım 6: $R1 \leftarrow R1 - 4R2$

2. sütunda pivotun üstünü sıfırlıyoruz.

1	0	−1	0
0	**1**	2	0
0	0	0	0

Kırmızı hücre pivotu, renkli satır o adımda değişen satırı gösteriyor. Dikey çizginin sağı sağ taraftır.

Serbest değişkene 1 verip pivot değişkenleri çözünce sıfır olmayan bir çözüm çıktı:

> $c = (1, -2, 1)$
> $v_1 - 2 \cdot v_2 + v_3 = (0, 0, 0)$

Bağıntıyı v_3 için çöz: v_3 diğerlerinin doğrusal birleşimidir, yani yeni bir yön katmaz. Burada $v_2 = (v_1 + v_3)/2$: ortadaki vektör, diğer ikisinin tam ortalaması (ardışık sayılar eşit aralıklı).

> $v_3 = -v_1 + 2 \cdot v_2$

rank = 2 ⇒ vektörlerin hepsi $\mathbb{R}^3$ içinde orijinden geçen bir düzlem üzerinde. Gerilen uzayın boyutu 2; bir baz: $\{v_1, v_2\}$ (pivot sütunlarına karşılık gelen orijinal vektörler).

▼ Sonucu göster

Doğrusal BAĞIMLI.
rank = 2 < 3 = vektör sayısı: serbest değişken var, sıfır olmayan katsayılarla sıfır elde edilir.
Sıfır olmayan bir bağıntı $(c_1, \ldots, c_k) = (1, -2, 1)$: 1·v1 − 2·v2 + 1·v3 = 0

Gerilen uzay için baz: {v1, v2} = (1, 2, 3), (4, 5, 6)

Boyut = rank = 2. Baz, indirgenmiş matristen değil, pivot sütunlarına karşılık gelen ORİJİNAL vektörlerden seçildi.

Üç vektör $\mathbb{R}^3$'te (sayı = boyut) olduğu hâlde bağımlı: vektör sayısı tek başına ölçüt değildir, rank'e bakılır.

`core/analysis.py (satır 435-476)`

```python
def dependence_analysis(vectors: Sequence[Sequence[Fraction]]) -> Dict:
    sonuc = null_space(vectors)
    elim = sonuc["elim"]
    temel = sonuc["basis"]
    k = len(vectors)
    pivotlar = elim.pivot_columns()
    bagimsiz = not temel
    iliskiler = []
    for d in temel:
        kontrol = arithmetic.linear_combination(vectors, d)
        if any(x != 0 for x in kontrol):  # pragma: no cover - motor güvencesi
        iliskiler.append(format_vector(d))
    return { ... }
```

null_space ile $c_1v_1 + c_2v_2 + c_3v_3 = 0$ sisteminin temel çözümlerini bulur. Bulunan her bağıntı gerçekten sıfır veriyor mu diye linear_combination ile yeniden kontrol edilir (motor güvencesi). rank = 2 < 3 = vektör sayısı ⇒ bağımlı. Bağıntı: $v_1 - 2v_2 + v_3 = 0$.

Geren kümeden baz seçmek

Soru: S = {(1, 2, 1), (2, 4, 2), (0, 1, 1), (1, 3, 2)} kümesinin gerdiği alt uzay için S'nin içinden bir baz seçiniz ve boyutu bulunuz.

İstenen: Hangi vektörler fazladır; kalanlar bağımsız ve aynı uzayı geren bir küme oluşturur mu?

Adım 1: Başlangıç matrisi

1	2	0	1	0
2	4	1	3	0
1	2	1	2	0

Adım 2: $R2 \leftarrow R2 - 2R1$

1. sütundaki 2 değerini sıfırlıyoruz.

1	2	0	1	0
0	0	1	1	0
1	2	1	2	0

Adım 3: $R3 \leftarrow R3 - R1$

1. sütundaki 1 değerini sıfırlıyoruz.

1	2	0	1	0
0	0	1	1	0
0	0	1	1	0

Adım 4: $R3 \leftarrow R3 - R2$

3. sütundaki 1 değerini sıfırlıyoruz.

1	2	0	1	0
0	0	**1**	1	0
0	0	0	0	0

Kırmızı hücre pivotu, renkli satır o adımda değişen satırı gösteriyor. Dikey çizginin sağı sağ taraftır.

Serbest değişkene 1 verip pivot değişkenleri çözünce sıfır olmayan bir çözüm çıktı:

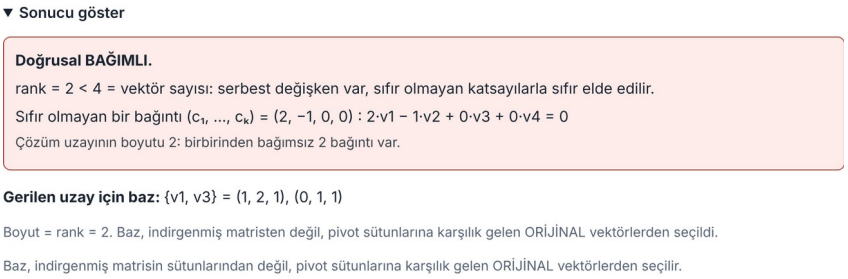
> $c = (2, -1, 0, 0)$
> $2 \cdot v_1 - v_2 = (0, 0, 0, 0)$

Bağıntıyı v_2 için çöz: v_2 diğerlerinin doğrusal birleşimidir, yani yeni bir yön katmaz.

> $v_2 = 2 \cdot v_1$

rank = 2 ⇒ vektörlerin hepsi $\mathbb{R}^3$ içinde orijinden geçen bir düzlem üzerinde. Gerilen uzayın boyutu 2; bir baz: $\{v_1, v_3\}$ (pivot sütunlarına karşılık gelen orijinal vektörler).

> $v_2 = 2 \cdot v_1$
> $v_4 = v_1 + v_3$



core/analysis.py (ilgili satırlar)

```python
# core/analysis.py → dependence_analysis (ilgili satırlar)
pivotlar = elim.pivot_columns()
...
"basis_indices": pivotlar,
"basis": [format_vector(vectors[c]) for c in pivotlar],   # orijinal vektörler
```

Aynı dependence_analysis fonksiyonu kullanılır. Baz, indirgenmiş matrisin sütunlarından değil; pivot sütun numaralarına karşılık gelen orijinal vektörlerden seçilir: vectors[c]. Bu soruda pivotlar 1. ve 3. sütun ⇒ baz {(1,2,1), (0,1,1)}, boyut 2.

Sonuç

Vektör uzayı, toplama ve skalerle çarpma altında kapalı olan ve V1-V8 kurallarını sağlayan bir kümedir. Sadece oklar değil, matrisler, polinomlar ve fonksiyonlar da vektör uzayı oluşturur.

Bir alt kümenin alt uzay olup olmadığına bakarken önce sıfır vektörünü kontrol etmek işi kolaylaştırıyor. Sıfır yoksa alt uzay değildir. Sıfır varsa toplama ve skalerle çarpma kapalılığına bakmak gerekiyor; sağlanmayan bir koşul için tek bir karşı örnek yeterli.

Span(S) her zaman bir alt uzaydır. Bir vektörün span içinde olup olmadığı Ax = b sisteminin çözümü olup olmadığına bakılarak, uzayın tamamının gerilip gerilmediği ise rank ile anlaşılıyor.

Vektörler bağımsızsa her sütunda pivot çıkıyor. Baz hem bağımsız hem de geren bir kümedir ve baz seçerken indirgenmiş matristeki değil, pivot sütunlarına karşılık gelen orijinal vektörleri almak gerekiyor.